

# Dataset and DataLoader in PyTorch

Mini-Batches, Sampling, Collation, and Efficient Training

## Abstract

A scalable training pipeline must separate how samples are obtained from how those samples are ordered, grouped, transformed, and delivered to optimization code. PyTorch expresses this separation through **Dataset**, which defines indexed sample access, and **DataLoader**, which turns that access into iterable mini-batches. These notes develop the complete path from manual mini-batch gradient descent to custom datasets, shuffling, samplers, collation, worker processes, loader parameters, and a batch-based breast-cancer classifier.

## Contents

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| <b>1</b>  | <b>Why mini-batches are needed</b>                    | <b>2</b>  |
| <b>2</b>  | <b>A manual mini-batch loop</b>                       | <b>2</b>  |
| <b>3</b>  | <b>Separation of concerns: Dataset and DataLoader</b> | <b>4</b>  |
| <b>4</b>  | <b>The Dataset contract</b>                           | <b>4</b>  |
| 4.1       | Synthetic classification data . . . . .               | 4         |
| 4.2       | Implementing indexed access . . . . .                 | 6         |
| 4.3       | Inspecting the dataset . . . . .                      | 6         |
| <b>5</b>  | <b>How DataLoader constructs a batch</b>              | <b>6</b>  |
| 5.1       | Creating and iterating over a loader . . . . .        | 7         |
| <b>6</b>  | <b>Per-sample transformations</b>                     | <b>7</b>  |
| <b>7</b>  | <b>Parallel loading with workers</b>                  | <b>8</b>  |
| <b>8</b>  | <b>Sampling strategies</b>                            | <b>9</b>  |
| <b>9</b>  | <b>Collation and variable-length samples</b>          | <b>9</b>  |
| <b>10</b> | <b>Important DataLoader parameters</b>                | <b>11</b> |
| <b>11</b> | <b>Complete breast-cancer mini-batch pipeline</b>     | <b>11</b> |
| 11.1      | Data preparation and loaders . . . . .                | 11        |
| 11.2      | Model, optimization, and evaluation . . . . .         | 13        |
| <b>12</b> | <b>Train/evaluation state and reproducibility</b>     | <b>14</b> |

|                                          |    |
|------------------------------------------|----|
| 13 Common pitfalls and diagnostic checks | 15 |
| 14 Review checklist                      | 15 |
| 15 Key takeaways                         | 15 |

## 1 Why mini-batches are needed

Suppose a dataset contains  $N$  training examples and a model has parameters  $\theta$ . Full-batch gradient descent computes one average loss over all examples,

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i), \quad \theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta).$$

Here  $x_i$  is the  $i$ th feature vector,  $y_i$  is its target,  $f_{\theta}$  is the model,  $\ell$  is the per-example loss,  $\eta$  is the learning rate, and  $\nabla_{\theta} \mathcal{L}$  is the gradient. The equation computes one update using every row. Its gradient is stable, but all examples needed by the forward and backward computations must be handled together, and only one parameter update occurs per epoch.

A mini-batch  $\mathcal{B}_k$  containing  $B$  examples instead uses

$$\mathcal{L}_k(\theta) = \frac{1}{|\mathcal{B}_k|} \sum_{i \in \mathcal{B}_k} \ell(f_{\theta}(x_i), y_i), \quad \theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}_k(\theta).$$

The symbol  $k$  identifies a batch,  $|\mathcal{B}_k|$  is its actual size, and  $\mathcal{L}_k$  is the batch-average loss. Each batch triggers an update, reducing working-memory demand and increasing update frequency. The gradient is noisier than a full-dataset gradient, but this stochasticity is often useful during optimization.

The number of batches is

$$K = \left\lceil \frac{N}{B} \right\rceil \quad \text{when the final incomplete batch is retained.}$$

For  $N = 1000$  and  $B = 100$ ,  $K = 10$ : each epoch performs ten forward–loss–backward–update cycles. For a 320-row training set with  $B = 32$ , exactly ten complete batches are produced.

### Intuition

A batch is the unit of computation and a sample is the unit of data. Full-batch gradient descent uses  $B = N$ ; stochastic gradient descent uses  $B = 1$ ; mini-batch gradient descent chooses a practical value between these extremes.

Large image collections illustrate the memory problem. A 500-row table may fit comfortably in memory, whereas many high-resolution images occupying several gigabytes may not. Loading and processing the entire collection simultaneously is therefore neither necessary nor desirable.

## 2 A manual mini-batch loop

The core algorithm can be written without `DataLoader`. The following reconstruction captures the demonstrated 320-row, batch-size-32 structure.

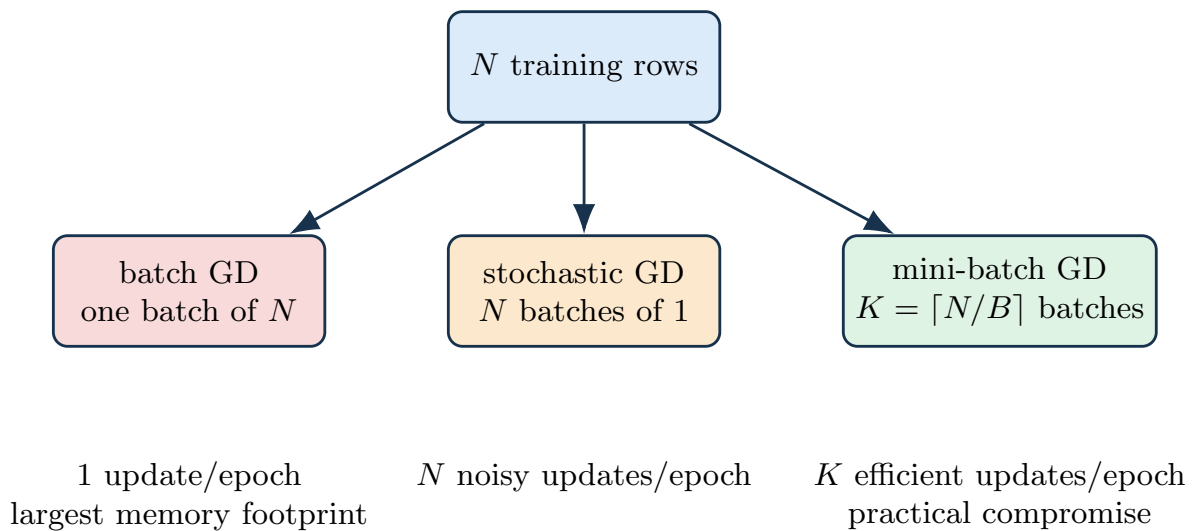


Figure 1: Gradient-descent variants differ in memory use, gradient noise, and the number of parameter updates per epoch. [Technical reference](#).

```

batch_size = 32

for epoch in range(epochs):
    # With 320 rows, start takes values 0, 32, ..., 288.
    for start in range(0, len(X_train_tensor), batch_size):
        stop = start + batch_size
        batch_features = X_train_tensor[start:stop]
        batch_labels = y_train_tensor[start:stop]

        optimizer.zero_grad()
        y_pred = model(batch_features)
        loss = criterion(y_pred, batch_labels)
        loss.backward()
        optimizer.step()
  
```

Listing 1: Manual slicing for mini-batch gradient descent

The outer loop counts epochs; the inner loop visits all mini-batches in one epoch. Slicing returns 32 rows for every batch when  $N = 320$ . Clearing gradients prevents unwanted accumulation, the forward pass produces predictions, the loss measures error, `backward()` computes gradients, and `step()` mutates parameters.

This code is mathematically adequate but operationally limited:

1. **No standard sample interface.** Tensor slicing assumes that all data is already materialized. It does not describe how to read a CSV row, decode an image, or navigate class folders.
2. **No natural transformation boundary.** Resizing, grayscale conversion, augmentation, text lowercasing, lemmatization, or stop-word removal needs an explicit home.
3. **Weak sampling control.** Ordered storage—for example, 1,000 dog images followed by 1,000 cat images—can create class-homogeneous batches unless indices are shuffled correctly.
4. **Manual batching and parallelization.** The code itself must manage index order, incomplete batches, collation, and parallel preparation.

### 3 Separation of concerns: Dataset and DataLoader

#### Definition

A map-style **Dataset** defines how an integer-like index maps to one sample. A **DataLoader** defines how indices are selected and grouped, how returned samples are collated, and how batches are exposed through iteration.

The abstractions decouple two questions:

- *What is sample  $i$ , and how should it be loaded or transformed?*
- *In what order and in what groups should samples reach the training loop?*

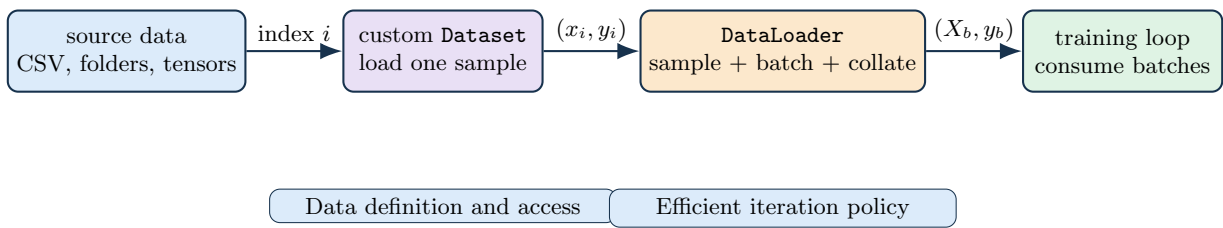


Figure 2: A Dataset owns sample access while a DataLoader owns iteration policy and batch construction. [Technical reference](#).

The training loop does not need to know whether a sample came from tensors, a CSV, image folders, a database, or another storage system. It only receives batches with a stable structure.

## 4 The Dataset contract

A custom map-style dataset normally inherits from `torch.utils.data.Dataset` and implements three methods:

`__init__` receives data, paths, labels, and optional transforms; it stores what later accesses need.

`__len__` returns the number of available samples  $N$ .

`__getitem__` receives index  $i$  and returns one sample, commonly the pair  $(x_i, y_i)$ .

### 4.1 Synthetic classification data

The first worked example contains ten rows, two features, and one binary target. Random values depend on scikit-learn and its random seed; shapes and types are deterministic.

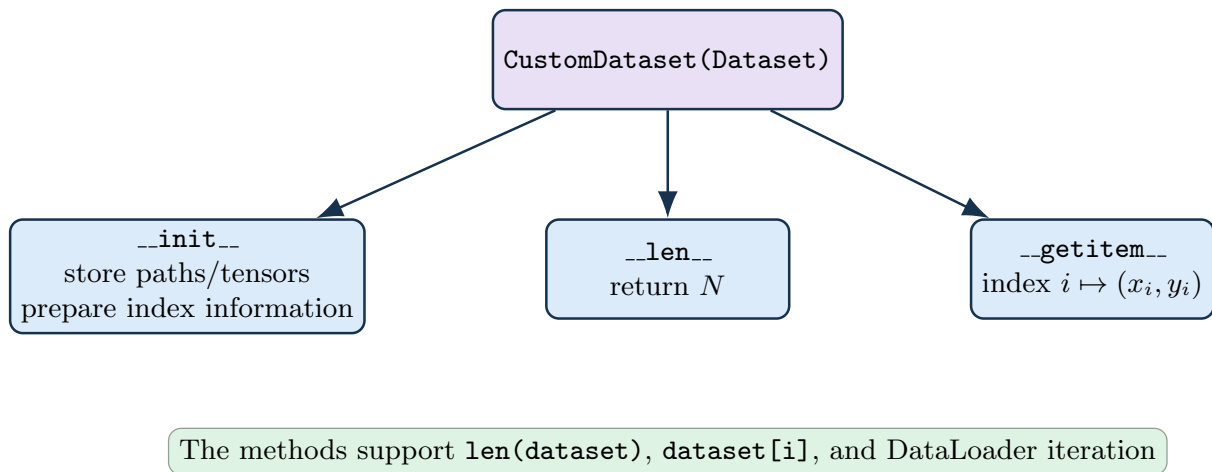


Figure 3: The three-method Dataset contract supports length queries, indexed retrieval, and `DataLoader` iteration. [Technical reference](#).

```

import torch
from sklearn.datasets import make_classification
from torch.utils.data import Dataset, DataLoader

X, y = make_classification(
    n_samples=10,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_clusters_per_class=1,
    random_state=42,
)

X = torch.tensor(X, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

print(X.shape)  # torch.Size([10, 2])
print(y.shape)  # torch.Size([10])
  
```

Listing 2: Create ten classification samples and convert them to tensors

Each row of  $X$  is a two-dimensional feature vector; each element of  $y$  is that row's class label. Conversion to `float32` makes the values compatible with ordinary neural-network layers and binary losses.

## 4.2 Implementing indexed access

```
class CustomDataset(Dataset):
    def __init__(self, features, labels, transform=None):
        self.features = features
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return self.features.shape[0]

    def __getitem__(self, index):
        feature = self.features[index]
        label = self.labels[index]
        if self.transform is not None:
            feature = self.transform(feature)
        return feature, label
```

Listing 3: A custom tensor-backed Dataset

`features.shape[0]` is the row count, so `len(dataset)` returns 10. Indexing both tensors with the same integer preserves feature–label alignment. The optional transform is applied after retrieval and before return, making preprocessing a property of sample access.

### Pitfall

Python’s special method names require exactly two leading and two trailing underscores: `__init__`, `__len__`, and `__getitem__`. A misspelling may allow class definition but prevents Python or DataLoader from invoking the intended protocol method.

## 4.3 Inspecting the dataset

```
dataset = CustomDataset(X, y)

print(len(dataset))          # 10
feature_0, label_0 = dataset[0]
feature_2, label_2 = dataset[2]

print(feature_0.shape)       # torch.Size([2])
print(label_0.shape)         # torch.Size([]): one scalar
print(feature_2.shape)       # torch.Size([2])
```

Listing 4: Construct the Dataset and retrieve individual rows

The source demonstration displayed particular random rows with values approximately  $[1.06, -0.97]$  and  $[-2.89, -2.89]$ . Such values are observations from one run, not universal expected output; generator version and seed determine exact coordinates. The invariant is that `dataset[i]` returns one two-value feature tensor and its aligned scalar target.

## 5 How DataLoader constructs a batch

For a ten-row dataset, the candidate index sequence is  $0, 1, \dots, 9$ . With shuffling enabled, a sampler may produce an order such as

4, 6, 1, 3, 8, 0, 9, 2, 7, 5.

A batch sampler groups that sequence according to  $B = 2$ :

$(4, 6), (1, 3), (8, 0), (9, 2), (7, 5).$

For the first pair, DataLoader requests `dataset[4]` and `dataset[6]`. The collate function combines the returned samples into tensors  $X_b \in \mathbb{R}^{2 \times d}$  and  $y_b \in \mathbb{R}^2$ . This repeats until all five batches are delivered.

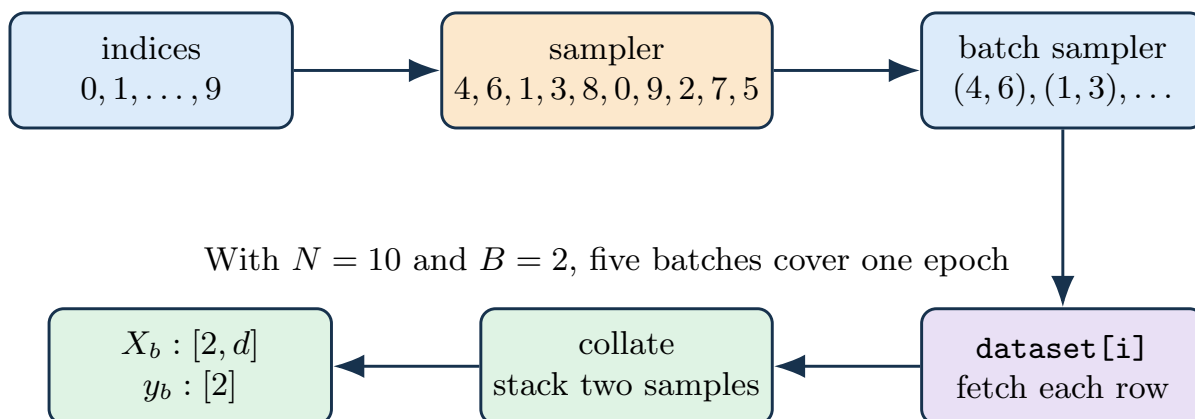


Figure 4: Sampling chooses index order, batch sampling groups indices, Dataset fetches rows, and collation stacks samples. [Technical reference](#).

## 5.1 Creating and iterating over a loader

```

data_loader = DataLoader(
    dataset,
    batch_size=2,
    shuffle=True,
)

for batch_number, (batch_features, batch_labels) in enumerate(data_loader, 1):
    print(
        batch_number,
        batch_features.shape,
        batch_labels.shape,
    )

# Five lines are printed. Every complete batch has:
# batch_features.shape == torch.Size([2, 2])
# batch_labels.shape   == torch.Size([2])

```

Listing 5: Create five shuffled batches of two samples

A `DataLoader` is iterable: a `for` loop repeatedly asks it for the next batch. With `shuffle=True`, row membership and order can change between epochs. With `shuffle=False`, the first batch contains rows 0 and 1, the second contains rows 2 and 3, and so forth.

### Shape check

For sample features of shape  $[d]$  and scalar labels, default collation with batch size  $B$  usually produces features of shape  $[B, d]$  and labels of shape  $[B]$ . The model or loss may require labels reshaped to  $[B, 1]$ .

## 6 Per-sample transformations

Transformations can be applied inside `__getitem__`, immediately before a sample is returned. Image transforms include `resize`, `grayscale conversion`, and `augmentation`. Text transforms include `lowercasing`,

lemmatization, tokenization, and stop-word removal.

```
class TransformDataset(Dataset):
    def __init__(self, features, labels, transform=None):
        self.features = features
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.features)

    def __getitem__(self, index):
        feature = self.features[index]
        label = self.labels[index]
        if self.transform is not None:
            feature = self.transform(feature)
        return feature, label

# Simple explanatory transform: standardize one feature vector
transform = lambda row: (row - row.mean()) / row.std().clamp_min(1e-8)
transformed_dataset = TransformDataset(X, y, transform=transform)
```

Listing 6: A Dataset that applies a supplied per-sample transformation

A transformation must preserve feature–target semantics. Random image augmentation belongs only in training data; deterministic normalization can be shared if its statistics were fitted without using test information.

## 7 Parallel loading with workers

With `num_workers=0`, the main process fetches and collates samples. Positive values create worker processes that can prepare different batches concurrently. One worker may fetch and transform one index chunk while another handles the next.

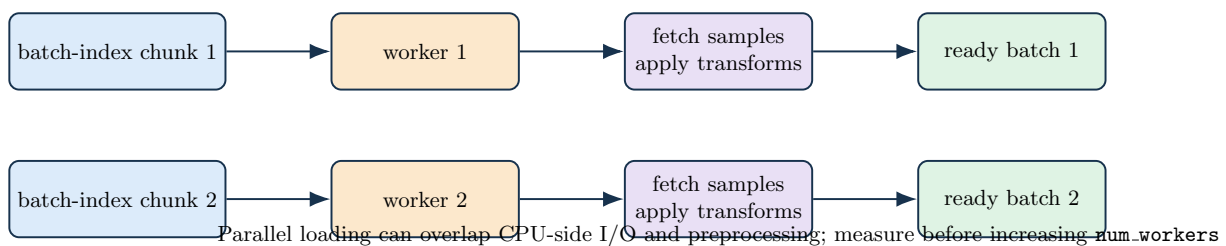


Figure 5: Multiple workers can prepare batches concurrently while preserving batch delivery through DataLoader. [Technical reference](#).

Worker counts such as 2, 4, or 8 are not universally optimal. The best value depends on CPU cores, storage speed, transform cost, operating system, dataset size, and GPU consumption rate. Too many workers can increase process startup cost, memory duplication, contention, or instability. Benchmark the complete input pipeline rather than assuming that a larger value is faster.

### Pitfall

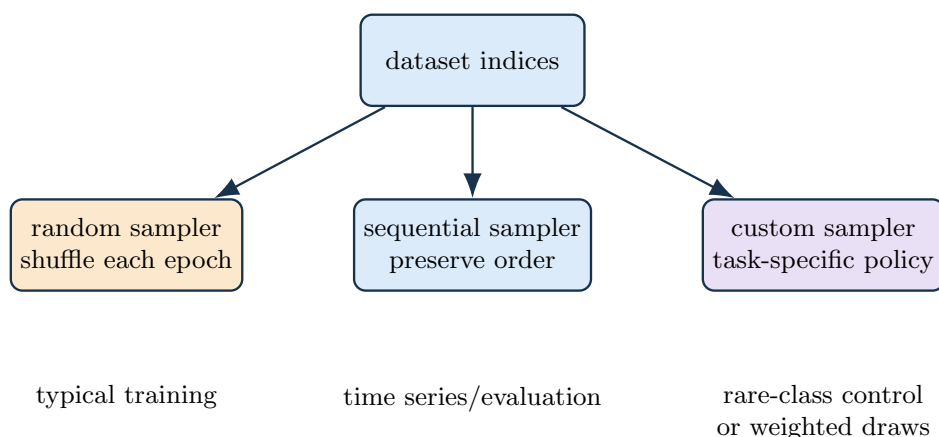
On Windows, worker processes use spawn semantics. Dataset objects and worker functions must be picklable, and executable scripts should protect top-level launching code with `if __name__ == "__main__":`. Notebook environments may behave differently; begin with `num_workers=0` when debugging.



## 8 Sampling strategies

A sampler determines the strategy for drawing dataset indices.

- A **random sampler** changes sample order and is typical for independent training examples.
- A **sequential sampler** preserves order; it is appropriate when order carries meaning, as in many time-series tasks, and is common for deterministic evaluation.
- A **custom or weighted sampler** implements task-specific selection, including rare-class oversampling or other constraints.



At 99:1 imbalance, naive random batches may omit the rare class

Figure 6: Random, sequential, and custom samplers encode different index-selection policies. [Technical reference](#).

For an imbalanced dataset with 99% class 1 and 1% class 0, unconstrained random batches may contain no class-0 examples. A weighted sampler can increase the probability of drawing rare examples. If the exact requirement is a fixed class composition in every batch, a custom *batch sampler* is more direct than relying on independent weighted draws.

### Pitfall

Do not pass both `shuffle=True` and an explicit `sampler`; both attempt to control index order. Likewise, `batch_sampler` is mutually exclusive with ordinary `batch_size`, `shuffle`, `sampler`, and `drop_last` settings.

## 9 Collation and variable-length samples

After sample retrieval, collation combines a list of samples into one batch. Default collation stacks equal-shape tensors. Variable-length text sequences cannot be stacked directly: shapes `[4]` and `[2]` differ. A custom `collate_fn` can pad shorter sequences and return lengths or a mask.

```

from torch.nn.utils.rnn import pad_sequence

# Each sample is (token_ids, label), where token_ids is one-dimensional.
def pad_collate(samples):
    token_sequences, labels = zip(*samples)
    lengths = torch.tensor([len(seq) for seq in token_sequences])
    padded = pad_sequence(
        token_sequences,
        batch_first=True,
        padding_value=0,
    )
    labels = torch.stack(labels)
    return padded, lengths, labels

text_loader = DataLoader(
    text_dataset,
    batch_size=2,
    collate_fn=pad_collate,
)

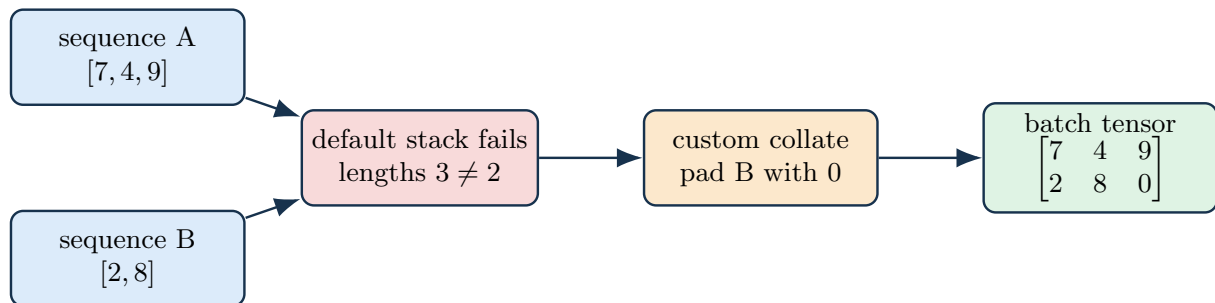
```

Listing 7: Custom collation for variable-length token sequences

If sequences  $[7, 4, 9]$  and  $[2, 8]$  share a batch, the output is

$$\begin{bmatrix} 7 & 4 & 9 \\ 2 & 8 & 0 \end{bmatrix}, \quad \text{lengths} = \begin{bmatrix} 3 & 2 \end{bmatrix}.$$

The zero is padding rather than observed data. Lengths or an attention mask prevent the model from treating padding as a real token.



Padding equalizes shape; masks or lengths should identify real tokens

Figure 7: Custom collation pads variable-length sequences so they can form a rectangular batch tensor. [Technical reference](#).

## 10 Important DataLoader parameters

| Parameter          | Meaning                            | Practical interpretation                                                                                                     |
|--------------------|------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>dataset</b>     | indexed sample source              | Supplies <code>__len__</code> and <code>__getitem__</code> .                                                                 |
| <b>batch_size</b>  | samples per batch                  | Controls memory use, update frequency, and gradient noise.                                                                   |
| <b>shuffle</b>     | randomize index order              | Usually true for training independent examples; normally false for evaluation or ordered data.                               |
| <b>num_workers</b> | loader worker processes            | Can overlap I/O and transforms; optimum is hardware- and workload-dependent.                                                 |
| <b>pin_memory</b>  | place CPU tensors in pinned memory | Can accelerate host-to-CUDA copies, especially with non-blocking device transfers; it does not by itself “activate the GPU.” |
| <b>drop_last</b>   | omit an incomplete final batch     | Gives fixed-size batches but discards up to $B - 1$ samples per epoch.                                                       |
| <b>collate_fn</b>  | sample-to-batch function           | Handles custom containers, padding, masks, and other batching logic.                                                         |
| <b>sampler</b>     | sample-index strategy              | Replaces simple sequential/random index selection.                                                                           |

Table 1: Frequently used DataLoader parameters and their roles.

For  $N = 32$  and  $B = 10$ , keeping the remainder produces batch sizes

$$10, 10, 10, 2.$$

Setting `drop_last=True` produces only the first three batches. Fixed batch sizes may matter when batch-dependent operations require stable statistics; however, discarding data should be an explicit trade-off rather than a default habit.

## 11 Complete breast-cancer mini-batch pipeline

The full application preserves a strict ordering: split first, fit preprocessing only on the training partition, transform both partitions, convert to tensors, construct datasets and loaders, then train and evaluate.

### 11.1 Data preparation and loaders

The exact official notebook cells were inaccessible, so the following code is a faithful reconstruction of the demonstrated workflow rather than a byte-for-byte reproduction.

```

import torch
import torch.nn as nn
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from torch.utils.data import Dataset, DataLoader

raw = load_breast_cancer()
X_train, X_test, y_train, y_test = train_test_split(
    raw.data,
    raw.target,
    test_size=0.2,
    random_state=42,
    stratify=raw.target,
)

# Fit only on training data to prevent test-set leakage.
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
y_test = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)

class CancerDataset(Dataset):
    def __init__(self, features, labels):
        self.features = features
        self.labels = labels

    def __len__(self):
        return len(self.features)

    def __getitem__(self, index):
        return self.features[index], self.labels[index]

train_dataset = CancerDataset(X_train, y_train)
test_dataset = CancerDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

```

Listing 8: Prepare breast-cancer data and build training and test loaders

The data has 30 measured features, so each ordinary training batch has  $X_b$  shape  $[32, 30]$ . Targets were explicitly reshaped to  $[N, 1]$ , so  $y_b$  has shape  $[32, 1]$  and matches one-neuron predictions. The final batch can be smaller.

Splitting before `fit_transform` is essential. If scaling statistics are computed from all rows, information from the held-out partition influences training features and produces optimistic evaluation.

## 11.2 Model, optimization, and evaluation

```

class CancerClassifier(nn.Module):
    def __init__(self, num_features):
        super().__init__()
        self.linear = nn.Linear(num_features, 1)

    def forward(self, features):
        return self.linear(features)  # logits, shape [B, 1]

learning_rate = 0.1
epochs = 100
model = CancerClassifier(X_train.shape[1])
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

for epoch in range(epochs):
    model.train()
    for batch_features, batch_labels in train_loader:
        optimizer.zero_grad()
        logits = model(batch_features)
        loss = criterion(logits, batch_labels)
        loss.backward()
        optimizer.step()

model.eval()
correct = 0
sample_count = 0
with torch.no_grad():
    for batch_features, batch_labels in test_loader:
        logits = model(batch_features)
        probabilities = torch.sigmoid(logits)
        predictions = (probabilities >= 0.5).float()
        correct += (predictions == batch_labels).sum().item()
        sample_count += batch_labels.numel()

accuracy = correct / sample_count
print(f"test accuracy: {accuracy:.3f}")

```

Listing 9: Train and evaluate a one-neuron classifier with mini-batches

The important state sequence for every training batch is:

1. `zero_grad()` clears gradients left by the previous batch;
2. the model maps  $X_b \in \mathbb{R}^{B \times 30}$  to logits  $z_b \in \mathbb{R}^{B \times 1}$ ;
3. binary cross-entropy with logits computes a scalar batch loss;
4. `backward()` populates parameter gradients;
5. `step()` applies the SGD update;
6. the next loader batch repeats the cycle.

For one example with logit  $z$ , target  $y \in \{0, 1\}$ , and probability  $p = \sigma(z)$ , binary cross-entropy is

$$\ell(z, y) = -[y \log p + (1 - y) \log(1 - p)], \quad p = \frac{1}{1 + e^{-z}}.$$

The target  $y$  selects the log-probability of the correct class. Lower loss means the assigned probability better agrees with the label. `BCEWithLogitsLoss` evaluates the sigmoid and cross-entropy together for numerical stability.

SGD updates each parameter component  $\theta_j$  by

$$\theta_j \leftarrow \theta_j - \eta \frac{\partial \mathcal{L}_b}{\partial \theta_j}.$$

Here  $\eta$  is the learning rate and  $\partial \mathcal{L}_b / \partial \theta_j$  measures how the current batch loss changes with parameter  $\theta_j$ . Repeating this update for every batch makes one epoch.

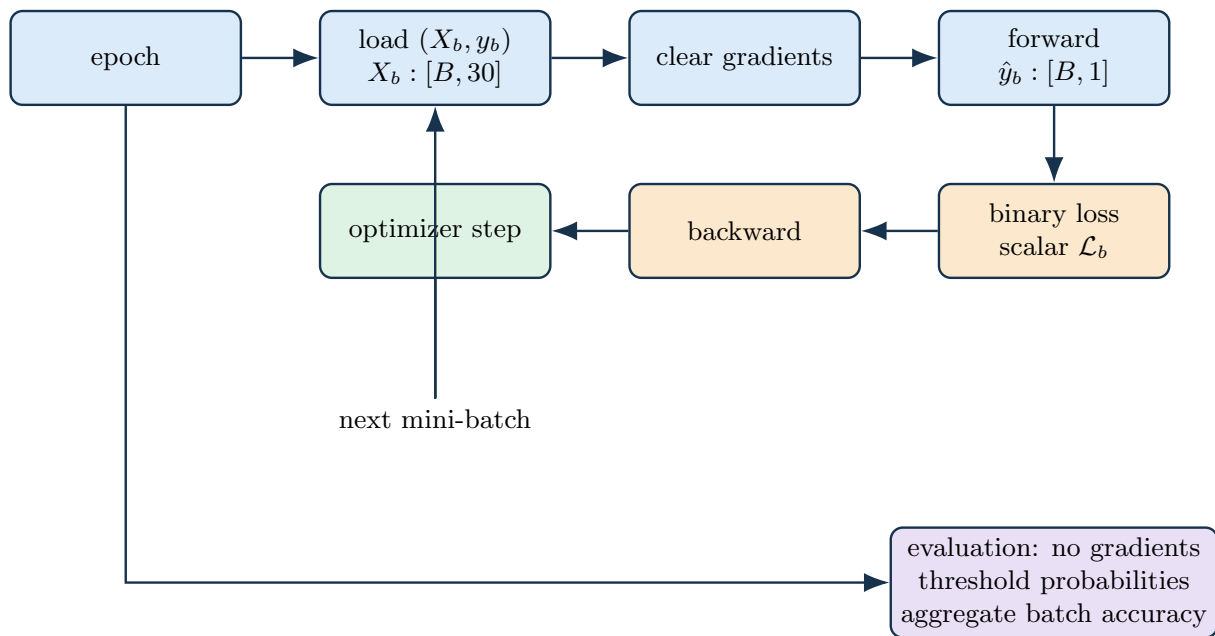


Figure 8: Training updates parameters batch by batch; evaluation disables gradient construction and aggregates predictions across test batches. [Technical reference](#).

The source workflow described averaging per-batch accuracies. The reconstruction instead accumulates correct predictions and divides by the total number of samples. This weighted aggregation is exact even when the final test batch is smaller; an unweighted average of batch accuracies would give the small final batch the same weight as a full batch.

### Shape check

Before calling a binary loss, verify that prediction and target shapes agree. For this model, logits and labels are both  $[B, 1]$ . A target of shape  $[B]$  should be reshaped with `view(-1, 1)` or `unsqueeze(1)`.

## 12 Train/evaluation state and reproducibility

Calling `model.train()` enables training behavior in modules such as dropout and batch normalization. On a separate line of the workflow, `model.eval()` switches them to evaluation behavior. The `torch.no_grad()` context separately disables gradient recording, reducing evaluation memory and work. Both state selection and gradient disabling belong in a robust evaluation pipeline.

Random shuffling should be controlled when reproducibility matters. A seeded `torch.Generator` can be passed to `DataLoader`, and worker seeding may be needed for random transforms. Reproducibility is not only a model concern; sampler order and augmentation randomness also affect the optimization path.

## 13 Common pitfalls and diagnostic checks

1. **Feature/label misalignment:** index both with the same  $i$  and shuffle indices, not feature and target arrays independently.
2. **Wrong dtypes:** neural-network features are commonly `float32`; class-index targets for multi-class cross-entropy are commonly integer `long`; binary targets should match the binary loss's expectations.
3. **Shape mismatch:** inspect one loader batch before training and compare it with the model's input/output contract.
4. **Preprocessing leakage:** fit scalers, encoders, and learned transforms only on training data.
5. **Unwanted transform randomness in evaluation:** use separate training and evaluation transform pipelines.
6. **Excessive workers:** benchmark worker counts and debug with zero workers first.
7. **Incorrect shuffle policy:** shuffle independent training examples, but preserve temporal order when order carries meaning.
8. **Unweighted metric averaging:** aggregate numerators and denominators across batches, especially when the last batch is smaller.
9. **Gradient accumulation:** clear gradients once per ordinary optimization step unless accumulation across several mini-batches is intentional.

## 14 Review checklist

A complete input pipeline should answer all of the following:

- What does one Dataset item contain, and what are its shapes and dtypes?
- Where are decoding and transformations performed?
- Which sampler determines order, and should order change each epoch?
- How does collation combine samples, especially variable-length ones?
- What are the batch shapes consumed by the model and loss?
- Is the incomplete final batch retained or dropped, and why?
- Do worker count and pinned memory improve measured throughput?
- Are training and evaluation loaders configured differently where appropriate?
- Are metrics aggregated over samples rather than naïvely over unequal batches?

## 15 Key takeaways

1. Mini-batches reduce memory demand and produce several parameter updates per epoch.
2. `Dataset` defines length and indexed sample retrieval; `DataLoader` defines iteration, batching, sampling, collation, and worker behavior.
3. Shuffling acts on indices, preserving feature–target alignment.
4. Per-sample transforms belong near sample retrieval; batch-level padding and packing belong in collation.
5. Samplers control which indices are drawn; custom policies can address imbalance or ordering constraints.

6. A robust training loop nests batch iteration inside epoch iteration and performs gradient clearing, forward computation, loss, backward computation, and parameter update in the correct order.
7. Evaluation should disable gradient recording, use evaluation state, preserve shape contracts, and aggregate metrics by sample count.